Agenda: Build First MCP Server and Client

- Setup - SDKs & MCPServer
- MCP CLI and package installation
- Stdio logging rule
- Hands-on lab setup
- Create and run the MCP server
- Verify using Inspector CLI
- Open and use the Inspector UI
- Use the server from Claude Desktop
- Write a Python client for the MCP server

Setup – MCPServer SDK

Python is Tier 1 (modelcontextprotocol/python-sdk, package mcp). The current release line is v2, which implements the latest MCP specification (2026-07-28).

Within it, **MCPServer** is the high-level decorator API — the fast path for tools/resources/prompts.

- Type hints → inputSchema — a: int becomes {"type": "integer"}
- Docstring → tool description — written once, surfaced to every client
- **Return value → tool result** — MCPServer wraps it as content[]
- Writing a Python function with type hints + docstring IS defining the protocol surface

```
@mcp.tool()
def multiply(a: float, b: float) -> float:
  """Multiply two numbers together and return the result."""
  return a * b
```

MCP CLI and Package Installation

The following installs the MCP package (v2) and the MCP CLI tools:

```
pip install "mcp[cli]"
```

MCP CLI:

- **mcp dev server.py** (npx @modelcontextprotocol/inspector python server.py — your development/debugging command. It launches the server together with the MCP Inspector. It also supports on-the-fly dependencies and live-reloading of local code: **mcp dev server.py --with pandas --with numpy** adds packages, and **--with-editable .** mounts your local code so changes show up without reinstalling.

- **mcp run server.py** — runs the server directly, as a plain process. **No Inspector,** no dependency management — it just imports your server object and starts it. The catch is that you're responsible for making sure all dependencies are available in the environment. Functionally this is close to just running **python server.py** yourself (assuming you have the if __name__ == "__main__": mcp.run()), so it's mostly a convenience wrapper.
- **mcp run server.py --transport streamable-http** — runs the same server over HTTP at **http://localhost:8000/mcp**. Note: **mcp run** imports your server object and calls **run()** itself, so it ignores your **if __name__ == "__main__"** block — the transport comes from the **--transport** flag (default: stdio).

The stdio logging rule (non-negotiable)

Your first servers use **stdio** (host launches the process; JSON-RPC over stdin/stdout). Therefore: ***STDIO servers must never write to stdout*** *— it carries the JSON-RPC messages; any stray write corrupts the protocol and the server dies "silently". (HTTP servers: stdout is fine.) — mcp-*

```
import sys, logging
print("hi")                    # ❌ corrupts a stdio server
print("hi", file=sys.stderr)   # ☑
logging.info("hi")             # ☑ (stderr/file)
```

A silent or instantly-dying first server is almost always a stray print()/banner on stdout. This is *the* classic bug.

Hands-On Lab

Prereqs: Python 3.10+ (python --version), uv (uv --version), Node.js 22.19+ (node --version, for the Inspector).

Setup: Install uv then restart your terminal:

```
# macOS/Linux
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Windows
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

Step 1 — Scaffold

```
# Option A — uv (recommended)
uv init mcp-first-demo
cd mcp-first-demo
python -m venv .venv
uv add "mcp[cli]"

# Option B — venv + pip
mkdir mcp-first-demo
cd mcp-first-demo
python -m venv .venv
.venv\Scripts\activate      # Windows PowerShell
source .venv/bin/activate   # macOS/Linux
pip install "mcp[cli]"
```

- Option A: uv run always uses the project's .venv — no activation needed.
- Option B: activate the .venv in every new terminal before running python or mcp.

Step2 - Create server.py

```
from mcp.server import MCPServer

# Create an STDIO/HTTP MCP server
mcp = MCPServer("Demo")

# Add an addition tool
@mcp.tool()
def add(a: int, b: int) -> int:
    """Add two numbers"""
    return a + b

@mcp.tool()
def subtract(a: int, b: int) -> int:
    """Subtract two numbers"""
    return a - b

@mcp.tool()
```

```
def multiply(a: float, b: float) -> float:
    """Multiply two numbers together and return the result."""
    return a * b

@mcp.tool()
def divide(a: float, b: float) -> float:
    """
    Divide a by b and return the result.
    Raises: ValueError: If b is zero
    """
    if b == 0:
        raise ValueError("Cannot divide by zero")
    return a / b

# Add a dynamic greeting resource
@mcp.resource("greeting://{name}")
def get_greeting(name: str) -> str:
    """Get a personalized greeting"""
    return f"Hello, {name}!"

# Main execution block - this is required to run the server
if __name__ == "__main__":
    mcp.run(transport="stdio")  #local
    #mcp.run(transport="streamable-http")  #remote
```

Note:

- The one thing to see here: greeting://{name} has a {placeholder}, so it's a **resource template** — reading greeting://Ada returns Hello, Ada!.
- The one thing to see: divide **raises ValueError** on zero — MCPServer turns the exception into a tool result with **isError: true**, so the server keeps running.

Step 3 - Run it:

```
uv run server.py
```

Note: It "hangs" waiting for JSON-RPC on stdin; Use Ctrl+C to stop).

Step 4 – Run the Inspector (no install needed)

Requires Node.js 22.19 or newer (node --version). npx downloads the Inspector on first use.

OR

First Install latest version of Node

npm install --global @modelcontextprotocol/inspector@latest

Verify with the Inspector CLI:

npx @modelcontextprotocol/inspector --cli uv run server.py --method **tools/list** # Python executes the file directly

OR

npx @modelcontextprotocol/inspector --cli mcp run server.py --method **tools/list** #The MCP CLI imports the file, finds its server object, and calls .run()

Note: mcp run imports the server object and calls run() itself, so it ignores your __main__ block — it uses stdio unless you pass **--transport streamable-http.**

npx @modelcontextprotocol/inspector --cli uv run server.py --method **tools/call** --tool-name add --tool-arg a=1 --tool-arg b=2

npx @modelcontextprotocol/inspector --cli uv run server.py --method tools/call --tool-name divide --tool-arg a=1 --tool-arg b=0

The result comes back with `**"isError": true**` — the server keeps running.

Open the Inspector UI

Launch the Inspector UI

mcp dev server.py

OR

npx @modelcontextprotocol/inspector python server.py

OR

Add to Workspace pyproject.toml

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"


[project]
name = "mcp-server-demo"
```

```
version = "0.1.0"
description = "Demo MCP server"
requires-python = ">=3.10"
dependencies = [
  "mcp[cli]>=2.0.0",
]

[tool.setuptools]
py-modules = ["server"]
```

then RUN

```
mcp dev server.py --with-editable d:/demos/mcp-server
```

Edit the server.py (change anything), **Refresh the browser** and note the change is reflected without restarting server.py

Connect and test tools and Resource Template

In the Inspector, open Server Settings and set Protocol Era = Auto, then click Connect. Open the Protocol tab: the client starts with server/discover, every request carries a _meta block (protocol version, client capabilities) and every result carries resultType.

To Debug MCP Server with Break point:

Change the main as below:

```
if __name__ == "__main__":
  #mcp.run(transport="stdio")  #local
  mcp.run(transport="streamable-http", port=8000)  #remote
```

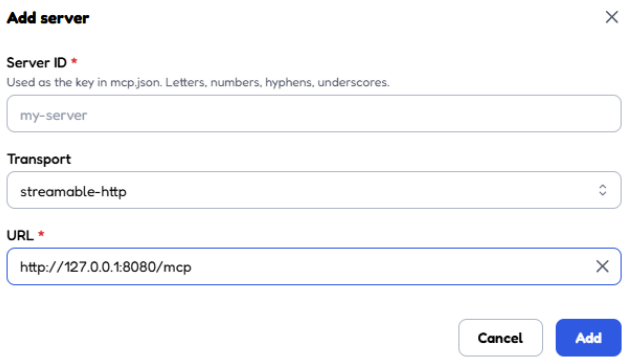
In Terminal 1:

```
uv run server.py
```

In Terminal 2:

```
npx @modelcontextprotocol/inspector
```

In Browser: open the URL printed in Terminal 2 (it includes a session token) → Add Server → Transport: Streamable HTTP → URL:



Note: the URL is http://localhost:8000/mcp (the port passed to mcp.run). In v2 the port is set in run(), not in the MCPServer(...) constructor.

Connect and test the method. Put a break point in Python Code to break at that point while calling the method from MCP client in MCP Inspector.

Use Server from Claude Desktop

The following command registers the server into the Claude Desktop config file so the app can launch it automatically.

```
mcp install server.py
```

OR

Manually Add to claude_desktop_config.json

```
"mcpServers": {
  "Demo": {
    "command": "uv",
    "args": [
      "run",
      "--frozen",
      "--with",
      "mcp[cli]==2.0.0",
      "mcp",
      "run",
      "D:\\Demos\\mcp-server\\server.py"
    ]
  }
}
```

Use the absolute path to uv (which uv on macOS, where uv on Windows, e.g. C:\\Users\\you\\.local\\bin\\uv.exe in JSON) — Claude Desktop does not always see your terminal's PATH. Then fully quit and restart Claude Desktop.

Writing a Python Client to use the MCP Server.

stdio_client.py

```
import sys
import anyio

from mcp import Client, StdioServerParameters

async def main() -> None:
    # Launch the local MCP server over stdio (sys.executable = this Python).
    params = StdioServerParameters(command=sys.executable, args=["server.py"])

    # Client discovers the server and adds _meta to every request.
    async with Client(params) as client:

        result = await client.call_tool("add", {"a": 5, "b": 7})
        print("add(5, 7) =", result.content[0].text)

        greeting = await client.read_resource("greeting://Alice")
        print("greeting://Alice =", greeting.contents[0].text)

        bad = await client.call_tool("divide", {"a": 1, "b": 0})
        print("divide(1, 0) is_error =", bad.is_error, "-", bad.content[0].text)

if __name__ == "__main__":
    anyio.run(main)
```

Key difference: tools return **.content**, resources return **.contents** (plural).

Result fields in SDK v2 use snake_case: **result.content**, **result.structured_content**, **result.is_error**, **tool.input_schema**.

If Streamable HTTP is used:

http_client.py

```
import anyio

from mcp import Client

async def main() -> None:
  # Default streamable-http endpoint is /mcp on port 8000.
  async with Client("http://localhost:8000/mcp") as client:
    result = await client.call_tool("add", {"a": 5, "b": 7})
    print("add(5, 7) =", result.content[0].text)

    greeting = await client.read_resource("greeting://Alice")
    print("greeting://Alice =", greeting.contents[0].text)

if __name__ == "__main__":
  anyio.run(main)
```

The pattern is identical — only the target passed to Client changes: StdioServerParameters for a local process, a URL for a remote server.

Terminal 1: Run the Server

```
uv run mcp run server.py --transport streamable-http
```

Terminal 2: Run the Client

```
uv run http_client.py
```

1. **This tool has four problems. Find them.**

```
@mcp.tool()
def orders(action: str, order_id: str, refund_cents: int = 0) -> dict:
  """Orders."""
  print("looking up", order_id)
  row = db.execute(f"SELECT * FROM orders WHERE id = {order_id}").fetchone()
  if action == "refund":
    db.execute("INSERT INTO refunds ...")
  return dict(row)    # includes email and phone
```